

1. Vibe 证明

- 接下来我将进行《自然语言处理》这门课的A2作业：Week3随堂Vibe实验：句法双引擎透视仪与“歧义侦探”。在Week3的理论课中，我学习了句子的两种拆解方式：成分句法（Constituency Parsing）关注短语如何嵌套，依存句法（Dependency Parsing）关注词与词之间的修饰与支配关系。讨论让人类和机器都头疼的结构歧义（Attachment Ambiguity）。因此我需要造一台能在网页上运行的“句法透视仪”，共有两个任务需要完成。任务一：建造“双视角”句法透视仪（任务目标：快速搭建一个Web原型，能够同时展示一句话的“成分句法树”和“依存句法图”）。任务二：化身“歧义侦探”与关系提取器（任务目标：程序跑通后，真正的NLP实验才刚开始。我需要利用这个工具，去验证课件中提到的理论，并增加“依存弧特征提取”功能）。（请了解这个作业的任务背景，不需要回复我任何信息）

好的。

- ### GPT-5.2-Codex:

你可以这样运行

- 安装依赖（若未安装）：`pip install streamlit spacy benepar nltk svgling`
- 启动：`streamlit run app.py`

运行后网页会显示左右两栏：左侧依存句法 SVG，右侧成分句法树（优先 SVG，若未装 svgling 则显示文本树）。

如果你希望我把启动命令和依赖写到笔记本里，或做成按钮/上传文件版本，告诉我即可。

- 成分句法渲染失败：请确认已安装 benepar 与其模型。详细错误：No module named 'IPython'——运行报错了，提示 No module named 'IPython'，请教我如何创建虚拟环境并给出 requirements.txt。

GPT-5.2-Codex:

已把 IPython 加到 requirements.txt 里，避免 svgling 渲染时缺依赖。

建议用 Python 3.10/3.11 新建环境（避免 3.13 的兼容问题）

- Conda:
 1. `conda create -n nlp-a2 python=3.11 -y`
 2. `conda activate nlp-a2`
 3. `pip install -r requirements.txt`
- venv:
 1. `python3.11 -m venv .venv`
 2. `source .venv/bin/activate`
 3. `pip install -U pip`
 4. `pip install -r requirements.txt`

安装完成后再运行 `streamlit run app.py`。如果你希望我把这些步骤写进 A2.ipynb 的操作记录里，告诉我即可。

- 请进行UI优化：左右两栏太挤了，把它改成使用 Streamlit 的 st.tabs（标签页），一个标签叫『依存关系』，另一个叫『成分结构』。

GPT-5.2-Codex:

已把左右两栏改成标签页布局，使用 `st.tabs`，标签为“依存关系”和“成分结构”。改动在 app.py。

如果你想再加一个“对比视图”标签（同时显示两种结果但更高的纵向空间），我也可以帮你加。

- 现在我将开始完成任务二：现在我要在页面最下方增加一个功能区：『核心论元提取器』。
- 请阅读课件理论：依存句法中包含核心论元（如 nsubj 主语, dobj 直接宾语）。

- ### GPT-5.2-Codex:

如果你希望把表格改成 Markdown 形式，或增加“句子中存在歧义”的提示规则，我可以继续加。

1.2 源代码文件：app.py

file:///Users/yumilde/Documents/ %E2%80%8D%E2%99%A2%E2%84%B8/SUFE/2025-2026-2/自然语言处理/A2/A2.html

```

        """生成依存句法的 SVG 字符串。"""
        doc = nlp(text)
        # 说明: displacy.render 返回 SVG 字符串
        svg = displacy.render(doc, style="dep", jupyter=False,
options={"distance": 80})
        return svg

def build_constituency_tree(nlp, text: str):
    """生成成分句法树; 优先返回 svgling 图形, 否则返回文本树。"""
    doc = nlp(text)
    # 说明: benepar 会将 constituency 结果挂在扩展属性上
    sent = list(doc.sents)[0]
    tree = sent._.parse_string
    if svgling is not None:
        # 说明: svgling 支持从 nltk.Tree 构建 SVG
        from nltk import Tree

        nltk_tree = Tree.fromstring(tree)
        return svgling.draw_tree(nltk_tree)
    return tree

def extract_core_arguments(nlp, text: str):
    """抽取依存关系中的核心论元 (nsubj/dobj/pobj/ROOT)。"""
    doc = nlp(text)
    core_labels = {"nsubj", "dobj", "pobj", "ROOT"}
    rows = []
    for token in doc:
        if token.dep_ in core_labels:
            rows.append(
                {
                    "依存关系": token.dep_,
                    "词": token.text,
                    "词性": token.pos_,
                    "支配词": token.head.text if token.head is not
token else "-",
                }
            )
        )
    return rows

def main() -> None:
    st.set_page_config(page_title="句法透视仪", layout="wide")
    st.title("句法双引擎透视仪")

    text = st.text_input("输入一句英文: ", value=DEFAULT_TEXT)

    # 说明: 按需下载语言模型, 避免首次运行报错
    ensure_spacy_model("en_core_web_sm")
    if benepar is not None:
        ensure_benepar_model("benepar_en3")

    # 说明: 构建 spaCy + benepar 的流水线
    nlp = spacy.load("en_core_web_sm")
    if benepar is not None and "benepar" not in nlp.pipe_names:

```

```

nlp.add_pipe("benepar", config={"model": "benepar_en3"})

dep_tab, const_tab = st.tabs(["依存关系", "成分结构"])

with dep_tab:
    st.subheader("依存句法 (Dependency Parsing)")
    try:
        dep_svg = build_dependency_svg(nlp, text)
        st.components.v1.html(dep_svg, height=500,
scrolling=True)
    except Exception as exc:
        st.error(f"依存句法渲染失败: {exc}")

with const_tab:
    st.subheader("成分句法 (Constituency Parsing)")
    try:
        tree = build_constituency_tree(nlp, text)
        if svgling is not None and hasattr(tree,
"_repr_svg_"):
            # 说明: 在 Streamlit 中直接显示 SVG
            svg = tree._repr_svg_()
            st.components.v1.html(svg, height=500,
scrolling=True)
        else:
            # 说明: 若没有 svgling, 则展示多级文本树
            st.code(tree)
    except Exception as exc:
        st.error(
            "成分句法渲染失败: 请确认已安装 benepar 与其模型。"
            f"\n详细错误: {exc}"
        )

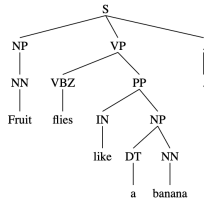
st.divider()
st.subheader("核心论元提取器")
try:
    core_rows = extract_core_arguments(nlp, text)
    if core_rows:
        st.dataframe(core_rows, use_container_width=True)
    else:
        st.info("未检测到 nsubj/dobj/pobj/R00T 相关依存关系。")
except Exception as exc:
    st.error(f"核心论元提取失败: {exc}")

if __name__ == "__main__":
    main()

```

2. 成功截图

成分句法 (Constituency Parsing)



核心论元提取器

依存关系	词	词性	支配词
nsubj	Fruit	NOUN	flies
ROOT	flies	NOUN	flies
pobj	banana	NOUN	like

3. 侦探报告

3.1 侦探任务 1：再探“介词短语附着歧义”

- 在成分树中，“with the telescope”是和“saw”组成了 VP，它在语法上被处理为状语 (Adjunct)，修饰动作。
- 在依存图中，“telescope”的箭头最终通过介词结构连向了“saw”。
- 在成分树中，PP 挂载在 VP 下而非宾语 NP 下；在依存关系中，动作的发起工具指向了动词。这说明模型将其识别为工具状语，选择的语义是：“用望远镜看”

3.2 侦探任务 2：果蝇与香蕉

- 在成分树中，flies 下方的标签是 VBZ，代表动词单数第三人称现在时，因此模型被骗了，它把 flies 标注成了动词（飞），而不是名词（苍蝇）。

3.3 在分析 "The boy saw the man with the telescope." 时，你的模型做出了怎样的“艰难决定”？结合课件 P48 提到的 PCFG（概率上下文无关文法），或者基于神经网络的方法，你认为模型为什么会偏向这种解释？

- 面对这个经典的歧义句，模型其实在进行一场关于归属权的概率博弈，也就是语言学中著名的介词短语附件 (PP-attachment) 难题。它必须在“用望远镜看”和“拿着望远镜的人”这两个高概率分支中做出抉择。从 PCFG 的逻辑来看，模型在训练语料中计算出的 NP → NP PP 规则权重可能恰好压过了 VP 分支；而在现代神经网络眼中，这更多是一场语义向量的拉力赛，由于“man”和“with”在物理位置上更近，模型为了降低全局损失，往往会产生一种就近挂载的结构偏见。模型根据统计直觉认定这种名词修饰结构在语言分布上更稳妥，从而在这个艰难的二选一中做出了现在的决定。